

closure_evidence_20260925

BOB-196 closure evidence — CM-PLUGIN-COUNT gate runtime fix

Revision: 1 Last modified: 2026-09-25T00:00:00Z

Summary

BOB-196 reported the CM-PLUGIN-COUNT pre-build gate (`scripts/pre_build/check_cm_plugin_count.sh`) took ~189s (measured 2026-08-25) because its marker-scanning loop iterates EVERY line of every governed document and spawns a subprocess pipeline per line — the classic per-line-subprocess shell trap. The item's own candidate fix (stated as *unverified*) was to pre-filter the governed documents with one `grep` for lines carrying the CM-PLUGIN-COUNT: marker BEFORE entering the per-line loop, leaving the per-line parsing pipeline — including the `-oE | wc -l` footgun-avoidance the in-code comment documents — completely unchanged. This evidence file verifies that candidate fix.

§11.4.6 honest note on the historical timing

The item's own measurement (2026-08-25) was `rc=0 wall=189s`. On THIS host, TODAY, the BEFORE measurement (below) was ~**34–38s**, not ~189s. This is reported as measured, not assumed — the exact wall-clock figure clearly depends on host/CPU/subprocess-spawn characteristics that differ between the 2026-08-25 measurement environment and this session's host. The root cause (per-line subprocess spawning against ~1853–1883 lines) and the fix are

identical regardless of the absolute number; only the absolute seconds differ. The **speedup ratio** (~3.2–3.4x) is the load-bearing evidence, not the absolute second count.

Governed document line counts, measured independently today (§11.4.6, not assumed from the item's 2026-08-25 figures):

CLAUDE.md: 515 lines
AGENTS.md: 697 lines
docs/features/Status.md: 671 lines

(item's 2026-08-25 figures: AGENTS.md 689 + CLAUDE.md 494 + Status.md 670 = 1853; today's total = 1883 — the documents drifted slightly in the intervening month, as expected for governed docs under active development.)

Fix applied — exact diff

File touched: scripts/pre_build/check_cm_plugin_count.sh (only file changed; tests/pre_build/test_check_cm_plugin_count.sh was read but NOT modified — see the honest note in the closing section).

```
diff --git a/scripts/pre_build/check_cm_plugin_count.sh b/scripts/pre_build/check_cm_plugin_count.sh
index bfd24f6..5503ad5 100755
--- a/scripts/pre_build/check_cm_plugin_count.sh
+++ b/scripts/pre_build/check_cm_plugin_count.sh
@@ -275,6 +275,21 @@ for doc in $GOVERNED_DOCS; do
     [[ -f "$path" ]] || continue

     # --- marker-based claims
     -----
+   # BOB-196: pre-filter to the lines that could possibly carry
+       a marker
+   # BEFORE entering the loop, instead of spawning ~6 processes
+       for EVERY
+   # line of the document. Safe by construction: every line the
+       loop body
+   # below can act on MUST contain the literal substring "CM-
+       PLUGIN-COUNT:"
+   # (a strict superset of the anchored `<!--[:space:]]*CM-
+       PLUGIN-COUNT:`
+   # pattern the loop body itself tests two lines down), so this
+       fixed-string
+   # pre-filter cannot skip a line the unfiltered loop would
+       have processed.
+   # A non-matching line was ALWAYS a silent no-op `continue` in
+       the original
+   # (nmark comes back 0, then `metric` comes back empty from
+       `sed`, so
```

```

+ # `[[-n "$metric" ]] || continue` fires with zero side
+   effects) – cutting
+ # it before the loop changes NO output, only how many times
+   the per-line
+ # pipeline runs (measured ~1853 iterations -> ~8 on this
+   repo, 2026-09-25).
+ # The per-line parsing logic itself – INCLUDING the `-oE | wc
+   -l`
+ # footgun-avoidance below, which is the actual correctness
+   property this
+ # gate depends on (§11.4.201(12)) – is UNTOUCHED, byte-for-
+   byte.
+   while IFS= read -r line; do
+       # Occurrence count, NOT `grep -c`. MEASURED on this host
+       (ugrep 7.8.4,
+       # 2026-08-21): `grep -coE` on this exact input returns 3
+       at top level
@@ -304,7 +319,7 @@ for doc in $GOVERNED_DOCS; do
+       elif [[ $VERBOSE -eq 1 ]]; then
+           echo "    ok $doc: $metric = $stated"
+       fi
-   done < "$path"
+   done < <(grep -F 'CM-PLUGIN-COUNT:' "$path" || true)

+   # --- legacy unmarked wording (the pre-fix B0B-149 shape)
+   -----
+   while IFS= read -r line; do

```

Scope note: the fix touches ONLY the outer iteration source of the first (“marker-based claims”) loop, exactly as the item specifies. The second, “legacy unmarked wording” loop (which scans for the deprecated ****N managed plugins**** prose form) is untouched — the item’s own measured process-count math (~1853 lines × ~6 procs = ~11,118 spawns) accounts only for the marker loop’s 6-process-per-line pipeline (printf | grep -oE | wc -l | tr -d=4, plus printf | sed -n = 2), so that loop is the one identified as the dominant cost and the one the item’s candidate fix targets. Every per-line command inside the marker loop body (the nmark computation via grep -oE | wc -l | tr -d, the metric extraction via sed -n, and the stated extraction via grep -oE) is byte-for-byte unchanged.

Safety argument for the pre-filter (independently verified, not merely asserted): every line the loop body can act on must contain the literal substring CM-PLUGIN-COUNT: — this is a strict superset of the anchored regex <!-- [[[:space:]]*CM-PLUGIN-COUNT: the loop body itself tests, so a fixed-string pre-filter on that substring cannot exclude any line the unfiltered loop would have processed. A non-matching line was always a silent no-op in the original (the nmark computation on such a line returns 0, metric extraction returns empty, and [[-n

"\$metric"]] || continue discards it with zero side effects) — so skipping it before the loop changes no output, only how many times the per-line pipeline runs.

Also independently verified: `grep -F '...' "$path" || true` feeding a `while ... done < <(...)` process substitution does NOT trip `set -euo pipefail` when the `grep` finds zero matches (tested with a synthetic zero-match fixture under an identical `set -euo pipefail` shell — script survived and printed its post-loop line).

Before / after wall-clock (pasted time output, same host, back-to-back)

BEFORE (pre-fix, two consecutive runs for stability)

```
$ time bash scripts/pre_build/check_cm_plugin_count.sh
...
PASS: CM-PLUGIN-COUNT – 8 documented count(s) match their
derivation

real    0m38.046s
user    0m12.718s
sys 0m45.502s

$ time bash scripts/pre_build/check_cm_plugin_count.sh > /dev/null
real    0m34.779s
user    0m12.702s
sys 0m42.565s
```

AFTER (post-fix, two consecutive runs for stability)

```
$ time bash scripts/pre_build/check_cm_plugin_count.sh
...
PASS: CM-PLUGIN-COUNT – 8 documented count(s) match their
derivation

real    0m11.241s
user    0m3.064s
sys 0m10.372s

$ time bash scripts/pre_build/check_cm_plugin_count.sh > /dev/null
real    0m11.210s
user    0m2.909s
sys 0m10.392s
```

Result: wall-clock dropped from ~34–38s to ~11.2s — a **~3.2–3.4x speedup**, achieved purely by cutting the marker loop’s ~1883 total-line iterations down to the handful of lines that actually carry a CM-PLUGIN-COUNT: marker (8 in this repository today), while leaving every other cost centre (the legacy-wording loop, array_count/needle_proven control-needle proofs, the find traversal for recursive/toplevel) untouched. The remaining ~11s is legitimate residual cost outside this item’s stated scope (the untouched legacy loop still walks all ~1883 lines at 2 processes/line, plus the control-needle proofs and find walk).

Mutation-still-bites proof

Two independent proofs, both showing the correctness-critical -oE | wc -l footgun-avoidance is genuinely exercised and untouched:

Proof 1 — the existing paired-mutation fixture suite, run against the fixed gate

```
$ time bash tests/pre_build/test_check_cm_plugin_count.sh
PASS: golden-good (all documented counts match derivation) (rc=0
as expected)
PASS: golden-bad-curated (doc says 7, array holds 3 – must FAIL)
(rc=1 as expected)
PASS: golden-bad-legacy (unmarked '**42 managed plugins**' – must
FAIL) (rc=1 as expected)
PASS: golden-bad-missing (mandatory curated marker absent – must
FAIL) (rc=1 as expected)
PASS: golden-bad-engines (4th engine on disk, doc says 3 – must
FAIL) (rc=1 as expected)
PASS: golden-good-utility (utility module must not move engine
count) (rc=0 as expected)
PASS: golden-good-carrier (unmarked numbers are carriers, not
counts) (rc=0 as expected)
PASS: golden-bad-multimarker (3 markers on one line, all counts
correct – must still FAIL) (rc=1 as expected)
PASS: real-tree (boba checkout after the B0B-149 fix) (rc=0 as
expected)
```

test_check_cm_plugin_count: PASS – CM-PLUGIN-COUNT honest across 8 fixtures + real tree (\$11.4.107(10))

```
real    0m12.873s
user    0m3.511s
sys     0m12.060s
EXIT_CODE=0
```

golden-bad-multimarker is the fixture engineered specifically to trip the footgun-avoidance: three CM-PLUGIN-COUNT markers crammed onto one line, all with individually-correct numbers, so the ONLY thing that can make it FAIL is the multi-marker-per-line detection (the `nmark -gt 1` check backed by `-oE | wc -l`). It still correctly FAILs (`rc=1`) after the fix.

Proof 2 (extra rigor, beyond the stated bar) — deliberately mutating the inner pipeline back to the naive form

To independently confirm the footgun-avoidance is load-bearing and that the outer pre-filter did not accidentally neuter it, the single correctness line was mutated in a throwaway copy back to the naive `grep -coE` form the in-code comment documents as broken on this host inside a `set -euo pipefail` subshell:

```
-      nmark="$(printf '%s' "$line" | grep -oE '<!--  
[[:space:]]*CM-PLUGIN-COUNT:' | wc -l | tr -d ' ' ||  
true)"  
+      nmark="$(printf '%s' "$line" | grep -coE '<!--  
[[:space:]]*CM-PLUGIN-COUNT:' || true)"
```

Both variants run against a fresh copy of the golden-bad-multimarker fixture (3 markers on one line, all correct numbers):

Mutated gate (naive grep -coE) — the mutation causes `nmark=1` inside the `set -euo pipefail` subshell (matching the exact context-dependent misreading the in-code comment documents), so the multi-marker-per-line check never fires; the gate silently falls through to a single greedy `sed` extraction (which happens to land on the last marker, `toplevel`), accepts it as correct, and the curated marker is never recorded as seen — masking the REAL defect and surfacing only an unrelated “mandatory curated marker absent” finding:

=== FINDINGS ===

CLAUDE.md: mandatory '<!-- CM-PLUGIN-COUNT: curated -->' marker is absent – the managed-roster count must be stated and guarded, not omitted

FAIL: 1 plugin-count divergence(s) (CM-PLUGIN-COUNT, BOB-149)
MUTATED_RC=1

Fixed gate (BOB-196 change, -oE | wc -l preserved) — correctly detects `nmark=3`, reports the REAL defect by name, and also reports the secondary “mandatory marker absent” consequence:

=== FINDINGS ===

CLAUDE.md: 3 CM-PLUGIN-COUNT markers on one line – put each on

its own line; this parser checks one per line and would leave the rest silently unchecked

CLAUDE.md: mandatory '<!-- CM-PLUGIN-COUNT: curated -->' marker is absent – the managed-roster count must be stated and guarded, not omitted

FAIL: 2 plugin-count divergence(s) (CM-PLUGIN-COUNT, BOB-149)

FIXED_RC=1

This proves the mutation genuinely changes gate behaviour (it is not a no-op) and that the BOB-196 fix preserves the exact correctness property the item required — the footgun-avoidance still catches and correctly diagnoses the multi-marker-per-line defect that the naive form would have masked.

Identical-8-counts-and-verdict proof

```
$ diff /tmp/before_stdout.log /tmp/after_stdout.log
```

(no output – files identical)

Both BEFORE and AFTER runs against the real repository produced byte-for-byte identical stdout:

check_cm_plugin_count – CM-PLUGIN-COUNT static pre-build gate (BOB-149)

root: /home/milosvasic/Projects/boba

derived (control-needle proven):

curated 43 (install-plugin.sh PLUGINS=() – the canonical managed roster)

bootstrap 12 (setup.sh PLUGINS=() – one-time-setup subset)

engines 43 (distinct engine modules on disk)

toplevel 35 (plugins/*.py – engines PLUS utility modules)

recursive 68 (plugins/**/*.py – also counts per-dir

variants)

PASS: CM-PLUGIN-COUNT – 8 documented count(s) match their derivation

Same 8 documented counts checked, same PASS verdict, same 5 derived metric values (curated=43, bootstrap=12, engines=43, topLevel=35, recursive=68), exit code 0 in both cases.

Invocation path verified

scripts/pre_build_verification.sh (read-only, NOT modified per this item's constraints) invokes the gate at its invariant 46 as:

```

PLUGINCNT_GATE="${PROJECT_ROOT}/scripts/pre_build/
    check_cm_plugin_count.sh"
...
bash "${PLUGINCNT_GATE}" >"${PLUGINCNT_LOG}" 2>&1 ||
    PLUGINCNT_EXIT=$?

```

i.e. `bash scripts/pre_build/check_cm_plugin_count.sh` with no arguments from the project root — exactly the invocation timed above.

Honest note on the paired §1.1 mutation test (task acceptance bullet)

A genuine, PRE-EXISTING paired-mutation / golden-good / golden-bad test file for this gate ALREADY EXISTED at `tests/pre_build/test_check_cm_plugin_count.sh` (314 lines, 8 fixtures + a real-tree smoke check, cross-referencing §11.4.1/§11.4.4/§11.4.6/§11.4.43/ §11.4.69/ §11.4.86/§11.4.107(10)/§11.4.108/§11.4.115/§11.4.135/§11.4.201/ §11.4.224/§11.4.227). It was NOT fabricated for this closure — it was located, read in full, and run unmodified both before and after the fix (see “Mutation-still-bites proof” above, Proof 1). It is fixture-based (golden-good/golden-bad document content driving the real gate) rather than a script-source mutation harness, but its golden-bad-multimarker fixture is precisely engineered to be a no-op unless the `-oE | wc -l footgun-avoidance` genuinely fires — functionally equivalent to a §1.1 paired mutation for THAT correctness property. This file was read-only for this item; it was not edited.

Constraints honoured

- No `git add / git commit / git push` executed.
- `docs/workable_items.db`, `docs/Issues.md`, `docs/Fixed.md`, and the `workable-items` CLI were not touched.
- `scripts/pre_build_verification.sh` and `scripts/commit-push-all.sh` were read-only (referenced for invocation-path confirmation, never edited).
- No file under `scripts/ownership_*.sh`, `scripts/lib/ownership.sh`, or `start.sh` was touched.
- Only `scripts/pre_build/check_cm_plugin_count.sh` and this new evidence file were written; `tests/pre_build/test_check_cm_plugin_count.sh` was read-only.